

HƯỚNG DẪN THỰC HÀNH

Broken Access Control - Khám phá điều khiển truy cập bị lỗi

Mức độ: Trung bình

An toàn ứng dụng Web

Kiểm thử xâm nhập

1. Giới thiệu Lab

Thông tin	Chi tiết
Tên Lab	web-brokenaccess
Mức độ	Trung bình
Công cụ sử dụng	OWASP ZAP, Firefox

Mục tiêu học tập

- Vận dụng OWASP ZAP để rà soát, chặn bắt và chỉnh sửa HTTP Request/Response nhằm đánh giá cơ chế phân quyền của ứng dụng web.
- Phân biệt và khai thác các dạng lỗ hổng Broken Access Control: Forced Browsing, Privilege Escalation, IDOR (Insecure Direct Object Reference) và Information Disclosure.
- Đánh giá khả năng leo thang đặc quyền thông qua việc chỉnh sửa trực tiếp HTTP Request (trường role, User ID, Authorization header...).
- Kiểm thử cơ chế xác thực và quản lý phiên theo các hạng mục OWASP WSTG: WSTG-ATHZ-01, WSTG-ATHZ-02, WSTG-ATHN-06, WSTG-ATHN-07.
- Đề xuất biện pháp khắc phục phù hợp cho từng lỗ hổng phân quyền phát hiện được trong quá trình kiểm thử.

Kỹ năng đạt được

- Kỹ năng sử dụng thành thạo các chức năng Spider, Intercept, Request Editor, History, Sites và Alerts của OWASP ZAP.

- Kỹ năng phân tích và chỉnh sửa HTTP Request/Response (Header, Body, Cookie, Authorization, JWT).
- Kỹ năng nhận diện và khai thác lỗ hổng SQL Injection phục vụ bỏ qua xác thực trong ngữ cảnh kiểm thử phân quyền.
- Kỹ năng lập báo cáo, đánh giá mức độ rủi ro và đề xuất biện pháp khắc phục lỗ hổng Broken Access Control.

2. Kịch bản

Tình huống:

Một doanh nghiệp phát triển website thương mại điện tử trên nền tảng OWASP Juice Shop. Dù đã triển khai đăng nhập, phân quyền và giao diện quản trị, đội ngũ phát triển vẫn lo ngại ứng dụng còn tồn tại lỗ hổng Broken Access Control. Sinh viên đóng vai trò một Chuyên gia kiểm thử xâm nhập (Penetration Tester), được cấp quyền truy cập như người dùng thông thường và cần xác định xem ứng dụng có để lộ thông tin quản trị, cho phép leo thang đặc quyền, bỏ qua xác thực hoặc giả mạo người dùng khác hay không.

Vai trò của sinh viên:

- Đóng vai người dùng thông thường, sử dụng OWASP ZAP để rà soát, chặn bắt và chỉnh sửa HTTP Request nhằm kiểm tra khả năng truy cập trái phép tới tài nguyên và chức năng quản trị.
- Kiểm thử cơ chế xác thực và quản lý phiên để xác định các lỗ hổng bỏ qua đăng nhập, leo thang đặc quyền và giả mạo người dùng khác (IDOR).
- Rút ra kết luận về nguyên nhân gây ra từng lỗ hổng Broken Access Control và đề xuất biện pháp khắc phục phù hợp.

Sơ đồ mạng



3. Kiến thức bổ trợ

Broken Access Control

Broken Access Control (OWASP Top 10) xảy ra khi ứng dụng không kiểm soát chặt chẽ quyền truy cập, cho phép người dùng thông thường truy cập tài nguyên hoặc thực hiện chức năng chỉ dành cho quản trị viên. Trong bài lab này, sinh viên sẽ lần lượt gặp bốn dạng phổ biến: **Forced Browsing** (truy cập trực tiếp đường dẫn không được liên kết công khai), **Privilege Escalation** (leo thang đặc quyền lên admin), **IDOR** (thao tác trên tài nguyên của người dùng khác qua tham số ID) và **Information Disclosure** (lộ dữ liệu nhạy cảm như hash mật khẩu).

HTTP Request, Session và JWT

Vì bài lab yêu cầu chỉnh sửa trực tiếp nhiều HTTP Request, sinh viên cần nắm rõ vai trò của từng thành phần trong Request (Request Line, Header, Body), cũng như cách Cookie, Session và JSON Web Token (JWT) được dùng để xác thực người dùng. Đó cũng chính là lý do vì sao chỉ cần thay đổi Header hoặc Body cũng có thể làm thay đổi quyền truy cập của ứng dụng.

Công cụ OWASP ZAP sử dụng trong bài

Chức năng ZAP	Mô tả tác dụng
Spider	Tự động thu thập URL, tài nguyên của ứng dụng để phát hiện đường dẫn hoặc chức năng không hiển thị trực tiếp trên giao diện.
Sites / History	Liệt kê các request đã ghi nhận được, dùng để tìm lại một request cụ thể cần chỉnh sửa hoặc phân tích.
Alerts	Hiển thị các cảnh báo bảo mật do ZAP phát hiện sau khi quét, ví dụ cấu hình sai liên quan đến trang quản trị.

Intercept	Chặn giữ một request trước khi gửi tới máy chủ để sinh viên có thể chỉnh sửa nội dung theo thời gian thực.
Request Editor	Cho phép mở lại, chỉnh sửa method, đường dẫn, Header và Body của một request rồi gửi lại (Resend) tới máy chủ.

4. Các nhiệm vụ của bài thực hành

Để khởi động bài thực hành, sinh viên dùng câu lệnh:

```
labtainer web-brokenaccess
```

Lưu ý: sinh viên sử dụng mã sinh viên của mình làm tên tài khoản khi được yêu cầu nhập email tại giao diện dòng lệnh, thông tin này được dùng phục vụ chấm điểm.

Tài khoản được cấu hình sẵn

Username	Password	Systems
admin@juice.org	admin123	Admin for web login
jim@juice.org	ncc-1701	web login
mc.safesearch@juice.org	Mr. N00dles	web login
Ubuntu	ubuntu	Student workstation, vulnerable server, Attacker workstation

Nhiệm vụ 1 - Xác nhận kết nối giữa máy trạm của sinh viên và máy chủ web

Bước 1: Tại giao diện dòng lệnh của máy sinh viên (web-brokenaccess), sử dụng lệnh

`ping` để xác nhận kết nối tới máy chủ web.

Điều kiện hoàn thành nhiệm vụ: Nhận được phản hồi từ địa chỉ `192.168.99.100`, xác nhận máy trạm và máy chủ web có thể liên lạc được với nhau.

Nhiệm vụ 2 - Mở Firefox và truy cập vào máy chủ web

Bước 1: Tại giao diện dòng lệnh của máy sinh viên (web-brokenaccess), gõ lệnh sau để mở trình duyệt Firefox:

```
firefox &
```

Bước 2: Trên Firefox, truy cập đường dẫn của ứng dụng web:

```
http://192.168.99.100:3000
```

Điều kiện hoàn thành nhiệm vụ: Truy cập thành công giao diện của ứng dụng OWASP Juice Shop trên Firefox.

Nhiệm vụ 3 - Khởi động và cấu hình OWASP ZAP

Bước 1: Tại giao diện dòng lệnh của máy sinh viên (web-brokenaccess), khởi động OWASP ZAP:

```
owasp-zap &
```

Bước 2: Khi ứng dụng OWASP ZAP mở lên và yêu cầu nhập thông tin, sinh viên chọn "yes, I want to persist this session with the name based on the current timestamp" rồi chọn **Start**.

Bước 3: Nếu xuất hiện hộp thoại "Manage Add-on", sinh viên chọn **Close** để tiếp tục.

Điều kiện hoàn thành nhiệm vụ: OWASP ZAP khởi động thành công và sẵn sàng để cấu hình làm proxy.

Nhiệm vụ 4 - Cấu hình Firefox Proxy để sử dụng OWASP ZAP

Mục tiêu: Cấu hình OWASP ZAP làm proxy trung gian để bắt và phân tích các gói tin HTTP/HTTPS truyền từ trình duyệt Firefox trên máy trạm của sinh viên. Thực hiện cấu hình trên trình duyệt Firefox theo các bước sau đây:

Bước 1: Trên Firefox của máy tính sinh viên, mở **Preferences**.

Bước 2: Trong ô tìm kiếm cài đặt, gõ `proxy`.

Bước 3: Trong mục **Proxy Network**, chọn **Settings**.

Bước 4: Chọn **Manual proxy configuration** (Cấu hình proxy thủ công).

Bước 5: Tại trường HTTP Proxy, nhập `127.0.0.1`, Port nhập `8080`.

Bước 6: Chọn thêm **Also use this proxy for HTTPS** (Sử dụng máy chủ proxy này cho tất cả các giao thức).

Bước 7: Chọn **OK** để chấp nhận các cài đặt.

Sau khi cấu hình xong, thực hiện các bước sau để xác nhận Firefox đang sử dụng ZAP làm proxy:

Bước 8: Làm mới trang web `192.168.99.100:3000`. Một cảnh báo bảo mật với thông báo "Kết nối của bạn không an toàn" sẽ xuất hiện.

Bước 9: Chọn **Advanced** để xem chi tiết chứng chỉ SSL (thường hiển thị lỗi `SEC_ERROR_UNKNOWN_ISSUER`), sau đó chọn **Add Exception** để chấp nhận chứng chỉ này.

Bước 10: Tại cửa sổ xác nhận, sinh viên tiếp tục chọn **Confirm Security Exception**.

Điều kiện hoàn thành nhiệm vụ: Firefox truy cập được ứng dụng web qua proxy của OWASP ZAP, tab **History** trong ZAP bắt đầu ghi nhận các request.

Nhiệm vụ 5 - Truy cập vào dữ liệu bị hạn chế

Mục tiêu: Mục tiêu của phần này là sử dụng công cụ Spider của OWASP ZAP để tự động thu thập các URL và tài nguyên của ứng dụng, qua đó xác định những đường dẫn hoặc chức năng không được hiển thị trực tiếp trên giao diện nhưng vẫn có thể được truy cập. Theo OWASP WSTG-ATHZ-01, việc lập danh sách các URL và thư mục của ứng dụng là một bước quan trọng trong quá trình kiểm thử phân quyền, giúp phát hiện các khu vực quản trị hoặc tài nguyên nhạy cảm bị lộ do cấu hình chưa phù hợp. Trong bài thực hành này, kết quả quét sẽ được sử dụng để tìm đường dẫn ẩn và cấu hình liên quan đến trang quản trị Admin.

Bước 1: Mở OWASP ZAP và thực hiện quét (Spider) trang web tại địa chỉ:

192.168.99.100:3000

Bước 2: Tại mục **Alerts**, tìm cảnh báo **Cross-Domain Misconfiguration**, từ đó tìm cấu hình liên quan đến trang quản trị.

Bước 3: Lưu kết quả quét của OWASP ZAP trên desktop với tên `report.html` (lưu toàn bộ báo cáo quét).

Điều kiện hoàn thành nhiệm vụ: Xác định được đường dẫn liên quan đến trang quản trị của ứng dụng thông qua kết quả Spider.

Câu hỏi 1:

Đường dẫn quản trị nào sinh viên tìm thấy qua kết quả Spider và vì sao đường dẫn này không nên bị lộ với người dùng thông thường?

Trả lời: _____

Nhiệm vụ 6 - Tự tạo tài khoản cho quản trị viên

Mục tiêu: Mục tiêu của phần này là kiểm tra xem ứng dụng có tồn tại lỗ hổng cho phép người dùng thông thường tạo tài khoản có quyền quản trị viên hay không. Theo hướng

dẫn OWASP WSTG-ATHN-06, cần phân tích các yêu cầu HTTP, API và tài nguyên phía máy khách để xác định xem các chức năng quản trị có bị lộ hoặc có thể bị truy cập trái phép thông qua việc sửa đổi các yêu cầu gửi đến máy chủ.

Bước 1: Sử dụng OWASP ZAP, tại trang đăng ký người dùng của ứng dụng, tạo một tài khoản mới với thông tin:

Email: `test@noemail.com`
 Password: `Password#1`
 Nhập lại mật khẩu: `Password#1`
 Câu hỏi bảo mật: Name of your favorite Pet? Trả lời: `Dogs`

Bước 2: Đăng nhập bằng tài khoản vừa tạo. Trong tab **History**, tìm request `GET` `http://192.168.99.100:3000/rest/user/whoami`, sao chép toàn bộ giá trị trường **Authorization** vào clipboard.

Bước 3: Tại tab **Sites**, tìm đường dẫn `http://192.168.99.100:3000` → `user` → `POST:Login`. Chọn chuột phải, chọn **Open/Resend with Request Editor**.

Bước 4: Trong Request Editor, sửa dòng đầu request thành `GET` `http://192.168.99.100:3000/api/Users`, xóa trường **Cookie** và dán giá trị **Authorization** vừa sao chép ở Bước 2. Chọn **Send** để xem danh sách toàn bộ tài khoản người dùng hiện có.

Bước 5: Chuyển lại request thành phương thức `POST`, đường dẫn `api/Users`, sau đó thêm vào phần Body trường mới `"role":"admin"` cùng với một địa chỉ email chưa từng được sử dụng. Chọn **Send** để gửi yêu cầu tạo tài khoản.

Bước 6: Đăng nhập bằng tài khoản vừa tạo và truy cập đường dẫn sau để xác minh tài khoản có quyền quản trị viên hay không:

`https://192.168.99.100:3000/#/administration`

Điều kiện hoàn thành nhiệm vụ: Tạo thành công tài khoản có quyền quản trị và truy cập được trang `/#/administration`.

Câu hỏi 2:

Vì sao việc chỉ thay đổi phương thức HTTP và đường dẫn API (không cần thay đổi giao diện) lại có thể dẫn đến leo thang đặc quyền trong trường hợp này?

Trả lời: _____

Nhiệm vụ 7 - Bỏ qua xác thực

Mục tiêu của nhiệm vụ này là tìm cách vượt qua yêu cầu đăng nhập của người dùng theo cách truyền thống, thông qua lỗ hổng SQL Injection tại chức năng đăng nhập.

Phương pháp 1: Vượt qua xác thực bằng hai dấu gạch ngang

Bước 1: Tại giao diện đăng nhập, nhập vào trường Email: `admin@juice.org'--` và nhập một mật khẩu bất kỳ.

Điều kiện hoàn thành: Đăng nhập thành công vào tài khoản `admin@juice.org` mà không cần biết mật khẩu.

Câu hỏi 3:

Sử dụng bảng tài khoản bên dưới, những tài khoản nào cũng dễ bị khai thác theo cách tương tự?

Trả lời: _____

Phương pháp 2: Bỏ qua xác thực bằng tên người dùng

Bước 1: Tại giao diện đăng nhập, nhập vào trường Email: `' OR email LIKE '%jim%'--` và chọn một mật khẩu bất kỳ.

Điều kiện hoàn thành: Xác định được liệu payload trên có cho phép đăng nhập hay không.

Câu hỏi 4:

Trong danh sách tài khoản bên dưới, những tài khoản nào cho phép đăng nhập bằng payload của Phương pháp 2?

Trả lời: _____

Danh sách tài khoản dùng để kiểm thử

Tài khoản	Dễ bị tấn công bởi (sinh viên tự điền)
admin@juice.org	
jim@juice.org	
bjoern.kimminich@gmail.com	
support@juice.org	
mc.safesearch@juice.org	
bender@juice.org	

Nhiệm vụ 8 - Giả mạo tài khoản

Mục tiêu: Mục tiêu của phần này là kiểm tra xem trang web có đảm bảo rằng mỗi phiên đăng nhập chỉ được sử dụng bởi đúng chủ sở hữu của phiên đó hay không. Theo OWASP WSTG-ATHZ-02, người dùng chỉ được phép thực hiện các thao tác và truy cập các tài nguyên thuộc quyền của mình. Trong bài thực hành này, sinh viên sẽ kiểm tra xem liệu có thể sử dụng thông tin xác thực hoặc phiên đăng nhập của một tài khoản khác để đăng bình luận hoặc thực hiện các hành động trái phép hay không.

Bước 1: Đăng nhập tài khoản `jim@juice.org` (mật khẩu `ncc-1701`).

Bước 2: Truy cập mục **Customer Feedback** trên ứng dụng.

Bước 3: Điền vào các trường **Comment**, **Rating** và mã CAPTCHA, nhưng chưa chọn nút **Submit**.

Bước 4: Trong OWASP ZAP, bật tính năng **Intercept**.

Bước 5: Trên Firefox, chọn **Submit**. OWASP ZAP sẽ hiển thị HTTP request đã bị chặn, gồm các trường:

`UserID` : ID của người dùng đăng bình luận

`captchaId` / `captcha` : câu hỏi và câu trả lời CAPTCHA

`comment` : nội dung bình luận

`rating` : đánh giá đã đăng

Bước 6: Sửa giá trị `UserID` thành ID của một người dùng khác (ví dụ tài khoản Admin), sau đó chọn gửi lại request.

Bước 7: Đăng nhập tài khoản `admin@juice.org` và kiểm tra kết quả tại đường dẫn sau để xác nhận bình luận đã được đăng dưới tên tài khoản nào:

`http://192.168.99.100:3000/#/administration`

Điều kiện hoàn thành nhiệm vụ: Bình luận được đăng dưới tên một tài khoản khác với tài khoản đã dùng để đăng nhập ban đầu.

Câu hỏi 5:

Vì sao lỗi hồng ở nhiệm vụ này được xếp vào nhóm IDOR (Insecure Direct Object Reference) thay vì Privilege Escalation?

Trả lời: _____

Nhiệm vụ 9 - Đăng nhập tài khoản và phá vỡ mật khẩu

Cơ chế xác thực phổ biến nhất và dễ quản trị nhất là mật khẩu tĩnh. Theo OWASP WSTG-ATHN-07, có thể chống lại các dạng tấn công mật khẩu phổ biến bằng cách đánh giá độ dài, độ phức tạp, sự tái sử dụng và yêu cầu thay đổi mật khẩu định kỳ. Nếu mật khẩu bị lộ dưới dạng hash, kẻ tấn công có thể tra cứu hash này qua rainbow table hoặc công cụ tìm kiếm.

Bước 1: Trong OWASP ZAP, tại tab **Alerts**, kiểm tra lại kết quả quét và tìm cảnh báo có liên quan đến đường dẫn:

```
https://192.168.99.100:3000/rest/user/authentication-details/
```

Bước 2: Tại tab **History**, tìm lại request tương ứng, chọn **Open/Resend with Request Editor** và chọn **Send** để quan sát nội dung phần Response trả về từ máy chủ.

Bước 3: Kiểm tra trường `password` trong phần Response nhận được. Ở các phiên bản Juice Shop trước đây, phần phản hồi có dạng tham khảo như sau:

```
[...]{ "id":1, "email":"admin@juice-sh.op",  
"password":"0192023a7bbd73250516f069df18b500" [...]
```

Bước 4: Đối chiếu với phản hồi thực tế nhận được ở Bước 2 và ghi nhận: nếu trường `password` không còn xuất hiện dưới dạng hash có thể tra cứu trực tiếp, đây là dấu hiệu cho thấy ứng dụng đã ẩn/mã hóa dữ liệu nhạy cảm này thay vì trả về nguyên văn như mô tả tham khảo ở Bước 3.

Điều kiện hoàn thành nhiệm vụ: Xác định được endpoint `authentication-details/` thông qua tab Alerts của OWASP ZAP, đồng thời nhận biết được rằng phản hồi hiện tại không còn lộ hash mật khẩu ở dạng có thể tra cứu như mô tả tham khảo.

Câu hỏi 6:

Nếu endpoint `authentication-details/` không còn trả về hash mật khẩu ở dạng lộ liễu như trước, theo em ứng dụng đã áp dụng biện pháp phòng thủ nào? Việc ẩn trường dữ liệu nhạy cảm trong response đã đủ để đảm bảo an toàn hay còn cần bổ sung biện pháp nào khác?

Trả lời: _____

